

Q1. Explain Windows Operating System Architecture with a neat diagram.

The Windows Operating System Architecture is a layered, modular design consisting of user-mode and kernel-mode components. It provides process management, memory management, device communication, security, and graphical interfaces. Understanding this architecture is crucial in Windows forensics to analyze system behavior, logs, registry, and artifacts.

1. Windows Architecture is Divided into Two Modes

A. User Mode

Runs applications and some system processes with limited privileges.

B. Kernel Mode

Low-level core of the OS; has full access to hardware and memory.

2. User Mode Components

1. Applications

User programs such as:

- Notepad
- Chrome
- MS Office
- Malware, if executed

Applications interact with the system through APIs.

2. Environment Subsystems

Provide compatibility layers for different environments:

- Win32 subsystem (most important)
- POSIX subsystem
- .NET CLR

The Win32 subsystem manages GUI, console, services, and graphics.

3. System Processes

Examples:

- csrss.exe → Client/Server Runtime Subsystem
 - winlogon.exe → Login management
 - services.exe → Service Control Manager
-

4. DLLs (Dynamic Link Libraries)

DLLs provide functions used by applications and system processes.

Common DLLs:

- kernel32.dll
 - user32.dll
 - advapi32.dll
 - gdi32.dll
-

3. Kernel Mode Components

1. Executive Layer

Contains core OS services:

- **Process Manager** → Handles processes, threads
 - **Memory Manager** → Manages virtual memory, paging
 - **I/O Manager** → Manages device I/O
 - **Security Reference Monitor** → Security policies, access checks
 - **Object Manager** → Manages internal OS objects
 - **Configuration Manager** → Manages registry
-

2. Kernel

Handles:

- Thread scheduling
 - Interrupts
 - Low-level processor operations
-

3. Hardware Abstraction Layer (HAL)

HAL isolates hardware complexity and allows Windows to run on multiple architectures seamlessly.

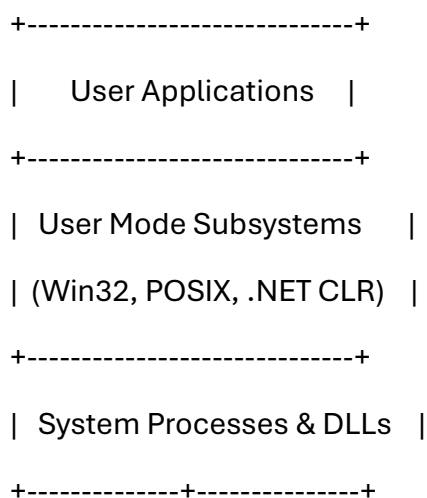
4. Device Drivers

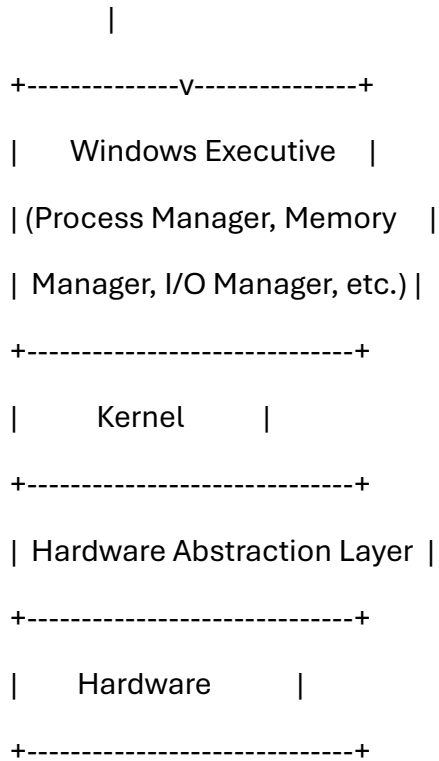
Drivers interact with devices such as:

- Disk drives
- Network adapters
- Input devices

Drivers often contain forensic artifacts, and may be exploited by rootkits.

Neat Diagram (Copy in Exam)





Forensic Relevance

- Rootkits often hide in kernel-mode components
- Memory dumps rely on understanding kernel structures
- Windows logs and registry originate from subsystems
- Process forensics require Executive and kernel knowledge

Conclusion

Windows architecture separates user and kernel operations, providing stability, security, and modularity. Forensic analysts must understand this structure to investigate system-level artifacts effectively.

Q2. Explain FAT File System Architecture and its forensic significance.

The **File Allocation Table (FAT)** file system is one of the oldest and simplest file systems, commonly used in USB drives, memory cards, and older Windows systems. It is still important in forensics due to its presence in removable media often used in cybercrimes.

1. FAT File System Structure

The FAT file system is divided into the following components:

1. Boot Sector (VBR – Volume Boot Record)

Contains:

- BIOS Parameter Block (BPB)
- File system type (FAT12, FAT16, FAT32)
- Cluster size
- Location of FAT tables
- Root directory information

Essential for identifying the file system layout.

2. FAT Tables (Usually FAT1 and FAT2)

These tables map cluster allocation.

FAT Table Purpose:

- Keeps track of used and unused clusters
- Identifies the chain of clusters belonging to a file
- Marks bad clusters

FAT redundancy (two copies) helps recover corrupted volumes.

3. Root Directory Region

- In FAT12/16: A fixed region
- In FAT32: A normal directory in data area

Directory entries contain:

- Filename (8.3 format)
 - Attributes (hidden, system, read-only)
 - Creation, access, modification times
 - Starting cluster
 - File size
-

4. Data Area

Actual file content is stored here in units called **clusters**.

2. FAT Cluster Chains

Files in FAT are stored in linked lists of clusters.

Example:

File A → Cluster 5 → 7 → 9 (end)

Broken links or orphaned clusters help identify deleted or fragmented files.

3. Forensic Significance of FAT

A. Easily Recover Deleted Files

When a file is deleted:

- First character of filename is set to 0xE5
- FAT chain is removed
- Data remains until overwritten

Tools can rebuild cluster chains.

B. Filename and Timestamp Artifacts

FAT stores three timestamps:

- Created
- Modified
- Accessed

Useful in timeline reconstruction.

C. Slack Space Analysis

Slack contains leftover data from previous files, useful for hidden evidence recovery.

D. Simplicity Aids Forensics

Minimal metadata means:

- Less anti-forensic complexity
 - Easy carving
-

E. Common on Removable Devices

Cybercriminals use USB devices frequently, making FAT forensics essential.

Conclusion

The FAT file system, though simple, contains valuable forensic artifacts such as deleted entries, slack space, timestamps, and cluster chains. Understanding FAT architecture helps recover hidden, deleted, and fragmented data effectively.

Q3. Explain NTFS File System Architecture and the role of the MFT.

NTFS (New Technology File System) is the default file system used by modern Windows systems. It is more robust and feature-rich than FAT, with advanced metadata handling, journaling, and security. NTFS is extremely important in forensic analysis because most Windows artifacts reside within NTFS structures.

1. NTFS Disk Layout

NTFS contains the following main structures:

1. Boot Sector

Contains:

- BIOS Parameter Block (BPB)
- Cluster size
- Location of MFT
- Volume information

2. Master File Table (MFT)

The most important component of NTFS.

Features of MFT:

- Every file and directory is stored as an entry in the MFT
- Each entry is typically **1024 bytes**
- Stores metadata and content of small files
- Includes system files (e.g., \$MFT, \$Bitmap, \$LogFile)

3. System Files

NTFS contains metadata files beginning with “\$”:

- **\$MFT** – Index of all files
- **\$MFTMirr** – Backup of first few MFT entries
- **\$Bitmap** – Cluster allocation map
- **\$LogFile** – Transaction journal
- **\$Boot** – Boot sector

- **\$Secure** – Security descriptors
- **\$Recycle.Bin** – Deleted file repository

These files contain rich forensic artifacts.

4. Data Area

Actual file contents stored in clusters.

2. Master File Table (MFT) – Detailed

Every file in NTFS is represented by an MFT Entry containing attributes:

Key Attributes:

1. \$STANDARD_INFORMATION

Contains:

- Timestamps (Creation, Modified, MFT Modified, Accessed)
 - File permissions
 - Ownership ID
-

2. \$FILE_NAME

Contains:

- File name
 - Parent directory reference
 - Additional timestamps
-

3. \$DATA

Contains file content.

For small files, data may be **resident** inside the MFT entry.

4. \$BITMAP, \$INDEX_ROOT, \$INDEX_ALLOCATION

Used for:

- Directory structure
 - Cluster allocation
 - Indexing
-

3. Forensic Significance of NTFS

A. MFT Contains Metadata for All Files

Even deleted entries remain until overwritten.

B. NTFS Stores Multiple Timestamps

This helps build accurate timelines.

C. Journaling via \$LogFile

Tracks file operations:

- Create
- Delete
- Modify
- Rename

Useful for intrusion analysis.

D. \$MFTMirr

Helps recover corrupted MFT.

E. Recovering Deleted Files

Even after deletion:

- MFT entry remains
 - Data clusters remain intact temporarily
-

Conclusion

NTFS is a powerful, metadata-rich file system. The MFT serves as the heart of NTFS, storing critical information about every file and directory. Understanding its structure is essential for recovering deleted files, reconstructing events, and identifying hidden or suspicious activity.

Q4. Explain how to recreate FAT and NTFS partitions and their forensic importance.

Recreating FAT and NTFS partitions is a crucial forensic technique used when partitions are deleted, corrupted, formatted, or intentionally damaged by attackers. The goal is to rebuild the original partition structure to recover data and reconstruct events.

1. Recreating FAT Partitions

FAT12, FAT16, and FAT32 partitions store important metadata in predictable locations, making reconstruction possible even after corruption.

A. Steps to Recreate a FAT Partition

1. Analyze the Volume Boot Record (VBR)

The FAT VBR contains:

- Bytes per sector
- Sectors per cluster
- Reserved sectors
- Number of FAT tables
- Total sectors

This information allows forensic tools to calculate partition boundaries.

2. Examine FAT Tables (FAT1 and FAT2)

Even if the partition table is erased, FAT tables may still exist.

Analysts can:

- Identify used and unused clusters
 - Trace cluster chains
 - Rebuild file allocation patterns
-

3. Recover Root Directory Entries

Even deleted entries (0xE5) provide:

- File names
- Attributes
- Starting cluster
- File size

This enables partial reconstruction of files.

4. Manual Carving and Tool-Based Reconstruction

Tools like:

- TestDisk
 - Autopsy
 - FTK
- can scan for FAT signatures and recreate partitions.
-

B. Forensic Importance of Recreating FAT Partitions

- Helps restore deleted partitions on USB drives/memory cards
- Rebuilds cluster chains for file carving
- Recovers metadata for timeline reconstruction

- Used in cases of accidental deletion or deliberate sabotage
-

2. Recreating NTFS Partitions

NTFS uses a structured metadata system, making it robust and recoverable even after corruption.

A. Steps to Recreate an NTFS Partition

1. Analyze the NTFS Boot Sector

It contains:

- MFT start location
- Cluster size
- Total sectors

This helps rebuild partition boundaries.

2. Locate the Master File Table (MFT)

Even if the partition table is deleted, the MFT often remains intact.

Analysts search for:

- Signature “FILE” or hex **46 49 4C 45**

Tools scan for MFT entries to determine partition layout.

3. Use MFT Mirror (\$MFTMirr)

NTFS keeps a backup of the first four MFT entries.

This is extremely useful for:

- Partition recovery
 - Rebuilding corrupted MFT
-

4. Extract Metadata Files (\$Bitmap, \$LogFile)

The \$Bitmap helps determine which clusters are allocated.
The \$LogFile provides transaction data for reconstruction.

5. Rebuild Partition Table Using Tools

Tools like:

- TestDisk
 - X-Ways
 - R-Studio
- can automatically rebuild NTFS partitions.
-

B. Forensic Importance of Recreating NTFS Partitions

- Restores access to damaged or formatted Windows partitions
 - Retrieves MFT entries for deleted files
 - Recovers crucial system files (\$LogFile, \$UsnJrnl)
 - Helps identify signs of anti-forensic techniques
 - Supports reconstruction of user activity timelines
-

Conclusion

Recreating FAT and NTFS partitions is essential in forensic investigations involving damaged, deleted, or overwritten data. Understanding partition structures enables investigators to recover lost evidence, reconstruct file systems, and detect malicious activities.

Q5. Explain the Windows Registry, its structure, and its forensic importance.

The **Windows Registry** is a hierarchical database used to store system settings, user configurations, application data, and hardware information. It is one of the richest sources

of forensic artifacts because nearly every action performed on a Windows system creates registry entries.

1. Structure of Windows Registry

The Registry is organized into **hives**, **keys**, **subkeys**, and **values**.

A. Registry Hives (Root Keys)

1. HKEY_LOCAL_MACHINE (HKLM)

Contains:

- Hardware details
 - Drivers
 - Installed software
 - System settings
-

2. HKEY_CURRENT_USER (HKCU)

Stores:

- User profiles
 - Desktop settings
 - Recently opened files
 - Application preferences
-

3. HKEY_CLASSES_ROOT (HKCR)

Stores file associations and COM object information.

4. HKEY_USERS (HKU)

Contains profiles of all users on the system, including default and guest profiles.

5. HKEY_CURRENT_CONFIG (HKCC)

Contains hardware configuration currently being used.

B. Registry Hive Files on Disk

Physical paths include:

- C:\Windows\System32\Config\SYSTEM
- C:\Windows\System32\Config\SOFTWARE
- C:\Users\<User>\NTUSER.DAT
- C:\Users\<User>\USRCLASS.DAT

These files can be extracted and analyzed offline.

2. Registry Data Types (Values)

- **REG_SZ** (string)
 - **REG_DWORD** (32-bit integer)
 - **REG_BINARY** (binary data)
 - **REG_MULTI_SZ** (multi-string)
-

3. Forensic Importance of Windows Registry

A. User Activity Traces

Registry stores user-specific artifacts such as:

- RecentDocs (recent files opened)
- RunMRU (commands executed)
- TypedPaths (file explorer history)
- ShellBags (folder view history)
- MountPoints2 (connected devices)

B. USB Device History

The Registry logs:

- USB vendor ID
- Device serial number
- Last connected time
- Volume name

This helps identify unauthorized data transfers.

C. Application Usage

Many applications store:

- Installation details
- Last run time
- Session information

Examples:

- Skype
 - Chrome
 - Microsoft Office
-

D. Persistence Mechanisms (Malware)

Malware often uses registry keys for persistence:

- HKCU\Software\Microsoft\Windows\CurrentVersion\Run
- HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run

These are common autostart locations.

E. Time-Based Forensic Analysis

Registry timestamps reveal:

- User login history
 - Application installs
 - System configuration changes
-

Conclusion

The Windows Registry is a powerful forensic artifact repository. Its detailed logs of user activity, system behavior, hardware usage, and application history make it one of the most important components in Windows forensic investigations.

Q6. Describe important Windows Registry artifacts used in forensic investigations.

Windows systems generate numerous registry artifacts that help investigators reconstruct user and system activities. These artifacts contain information about file access, program execution, device connections, network usage, and more.

Below are the most important forensic registry artifacts.

1. RecentDocs

Location:

HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\RecentDocs

Stores:

- Names and extensions of recently opened files
 - Helps identify user activity
-

2. MRU (Most Recently Used) Lists

RunMRU

Location:

HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\RunMRU

Stores:

- Commands typed in Run dialog
- Helps identify executed programs

OpenSaveMRU

Stores:

- Files opened/saved using application dialogs
-

3. ShellBags

Location:

HKCU\Software\Classes\Local Settings\Software\Microsoft\Windows\Shell\Bags

Stores:

- Folder view preferences
- Even for folders that no longer exist

Used to prove presence of deleted directories.

4. TypedURLs & TypedPaths

Store:

- URLs typed in Internet Explorer
- Paths typed in File Explorer

Useful in intrusion and browsing analysis.

5. UserAssist

Location:

HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist

Stores:

- Program execution history
- Counts how many times an application was run

- ROT13 encoded
-

6. Prefetch Interaction (Not Registry but Related)

Prefetch files (.pf) reveal:

- Last run time
- Number of executions
- DLL dependencies

Although not stored in registry, they complement registry evidence.

7. USB Device History

Location:

HKLM\SYSTEM\CurrentControlSet\Enum\USBSTOR

Stores:

- USB serial numbers
- Vendor and product IDs
- Connection timestamps

Critical for data exfiltration investigations.

8. LastVisitedMRU & LastVisitedPidlMRU

Store:

- Folder navigation history
 - URLs inside file dialogs
-

9. Installed Applications

Location:

HKLM\Software\Microsoft\Windows\CurrentVersion\Uninstall

Provides:

- Program names
- Installation dates
- Version info

Useful for detecting malicious or unauthorized software.

10. Network Profile Artifacts

Location:

HKLM\Software\Microsoft\Windows NT\CurrentVersion\NetworkList\Profiles

Provides:

- Connected Wi-Fi networks
- MAC addresses
- Connection history

Useful in physical location correlation.

Conclusion

Registry artifacts play a crucial role in forensic analysis by revealing detailed user interactions, execution history, and connected devices. They help investigators reconstruct timelines, detect suspicious activity, and gather evidence for legal proceedings.

Q7. Explain Windows Event Logs, their structure, and forensic importance.

Windows Event Logs are detailed records of system, security, and application activities. They are stored in **.evtx** (newer versions) or **.evt** (older versions) format and provide critical information for forensic investigations, incident response, and system monitoring.

1. Types of Windows Event Logs

A. System Logs

Contain information about:

- Driver failures
- System services
- Hardware issues
- Boot errors

Useful for detecting crashes or system manipulation.

B. Security Logs

Contain:

- Login attempts
- Logoff events
- Access to objects
- Privilege escalations
- Audit policy changes

Essential for intrusion analysis.

C. Application Logs

Generated by installed programs (e.g., SQL Server, Chrome, Antivirus).

Include:

- Application errors
 - Installation activities
 - Crashes
 - Program-specific events
-

D. Setup Logs

Contain installation activities such as:

- OS upgrades

- Hotfix installations
 - Service pack installations
-

E. Forwarded Events

Logs sent from other machines using event forwarding.

2. Structure of Event Log Files (.evtx)

Each log entry contains:

1. Event Header

- Record ID
- Timestamp
- Event level (Information, Warning, Error, Critical)
- Provider (source of event)

2. Event Data

Contains:

- User SID
- Process ID
- Thread ID
- Action or error description

3. XML Metadata (in .evtx files)

Modern logs use XML format for standardized parsing.

3. Forensic Importance of Event Logs

A. Authentication Traces

Security logs record:

- Successful logins (Event ID 4624)

- Failed logins (4625)
- Account lockouts (4740)

Helps track unauthorized access.

B. Intrusion and Malware Detection

Logs reveal:

- Service installation (7045)
 - Unexpected shutdowns
 - Privilege escalation attempts
 - Execution of suspicious processes
-

C. Timeline Reconstruction

Event logs provide timestamped entries that help investigators build a chronological timeline.

D. System Boot and Shutdown Information

Event IDs such as:

- 6005 → Event Log Started
 - 6006 → Event Log Stopped
 - 6008 → Unexpected Shutdown
-

E. Application Behavior Analysis

Crashes and abnormal behavior can indicate:

- Malware execution
- Exploit attempts
- Software tampering

F. Correlation with Other Evidence

Event logs complement:

- Registry records
- Prefetch files
- MFT timestamps

Together, they create a complete picture of system activity.

Conclusion

Windows Event Logs provide essential evidence for identifying unauthorized access, malware behavior, system failures, and user actions. Their structured, timestamped format makes them invaluable for forensic investigation and incident response.

Q8. Explain EVT and EVTX log formats and how forensic analysis is performed on them.

Windows uses two primary event log formats—**EVT** and **EVTX**. These logs contain important system, security, and application events. Understanding these formats is necessary for complete forensic examination.

1. EVT Log Format (Legacy Format)

Used in Windows XP and earlier.

Characteristics:

- Binary structure
- Limited size (~300 MB max)
- Less metadata than EVTX
- More prone to corruption

Structure:

- Header

- Array of event records
- Each record contains basic information such as timestamps, event ID, and message strings

Limitations:

- No XML structure
 - No checksums
 - Harder to detect tampering
-

2. EVTX Log Format (Modern Format)

Used in Windows Vista and above.

Characteristics:

- XML-based structure
- Bigger log size support
- Better metadata storage
- Recoverable even after corruption

Structure of EVTX File:

1. **File Header**
2. **Chunk Headers**
3. **Event Records in XML**
4. **Checksums for integrity**

Benefits:

- Supports event correlation
 - Enhanced security
 - Easier parsing with forensic tools
-

3. Forensic Analysis of EVT and EVTX Logs

Forensic analysis focuses on reconstructing user actions, identifying system changes, and detecting intrusions.

A. Extraction of Logs

Logs are stored in:

- C:\Windows\System32\winevt\Logs\ (EVTX)
- C:\Windows\System32\config (older EVT)

Investigators must:

- Copy logs using read-only access
 - Validate integrity with hashes
-

B. Viewing and Parsing Tools

Native Tools:

- Event Viewer
- Wevtutil.exe

Forensic Tools:

- FTK
 - X-Ways
 - LogParser
 - Chainsaw
 - EVTXtract (for corrupted logs)
-

C. Key Event IDs Used in Analysis

Authentication Events

- 4624 → Successful login
- 4625 → Failed login

Privilege Events

- 4672 → Admin privileges assigned

Process Execution

- 4688 → New process creation

Service Installation

- 7045 → New service installed

System Shutdown/Startup

- 6005 → Boot
 - 6006 → Shutdown
-

D. Detecting Tampering

Analysts look for:

- Gaps in event logs
 - Sudden clearing of logs (Event ID 1102)
 - Corrupted EVTX chunks
 - Suspicious timestamps
-

E. Correlation with Other Artifacts

Event logs must be correlated with:

- Prefetch files
- Registry keys
- MFT timestamps
- Web browser data

This results in an accurate timeline.

Conclusion

EVT and EVTX logs are vital sources of evidence. EVTX provides enhanced integrity, metadata, and XML structure, making it superior for forensic analysis. Both formats help investigators reconstruct authentication patterns, program execution, and system activity.

Q9. Explain the analysis of third-party Windows application logs with examples.

Third-party applications generate their own logs, which can reveal user activity, communication records, authentication events, error messages, and application behavior. These logs are extremely useful in forensic investigations, especially in cybercrime and insider threat cases.

1. Types of Third-Party Logs

A. Browser Logs

Stored by Chrome, Firefox, Edge, etc.

Contain:

- History
- Downloads
- Cookies
- Cache
- Autofill
- Login timestamps

Example (Chrome Artifacts):

- History SQLite DB
 - Cookies SQLite DB
 - Login Data DB
-

B. Messaging Application Logs

(WhatsApp Desktop, Skype, Telegram, Zoom)

Include:

- Chat history
- Timestamps
- File transfers
- Call logs
- Contacts

Example (Skype):

File: main.db contains full chat history.

C. Antivirus Logs

Contain records of:

- Malware detection
- Quarantine actions
- Scan history

Useful for identifying compromise attempts.

D. Cloud Application Logs

(OneDrive, Google Drive, Dropbox)

Contain:

- Upload/download activity
 - Sync history
 - Login IP addresses
-

E. Office Suite Logs (MS Office)

Store:

- Recently accessed files

- Document edits
- Print history

These support intellectual property theft investigations.

2. Techniques for Analyzing Third-Party Logs

A. Identifying Log Location

Most applications store logs in:

- %AppData%
 - %LocalAppData%
 - Program installation directories
-

B. Extracting SQLite Databases

Many modern apps use SQLite.

Tools:

- DB Browser for SQLite
 - Autopsy
 - X-Ways
-

C. Checking Timestamps

Investigators examine:

- Last accessed
- Modified
- Created timestamps

Time conversion (Unix epoch → readable date) may be required.

D. Keyword Searching

Search for:

- Communication keywords
- File names
- Usernames
- URLs

Useful in harassment and fraud cases.

E. Recovering Deleted Logs

Applications often store:

- Journals
- Temporary logs
- Sync files

Even deleted logs can be recovered.

3. Examples of Forensic Findings

Example 1: Browser Logs

A suspect visited illegal websites.

Evidence found in:

- Chrome History (History DB)
 - Cookies
 - Cache images
-

Example 2: Cloud Storage Logs

Files uploaded from a corporate PC.

Recovered through:

- OneDrive logs
- Sync metadata

- File version history
-

Example 3: Messaging Logs

An employee leaked data through Skype.

Chat history recovered from:

- main.db
 - Media files folder
-

Conclusion

Third-party application logs contain significant forensic evidence including communication, browsing, file transfer, login, and user activity patterns. Proper extraction, parsing, and correlation with system artifacts make them essential for modern forensic investigations.

Q10. Explain Prefetch files in Windows, their structure, and their forensic significance.

Prefetch files are Windows system files used to speed up application load times by storing information about programs that have been executed. They are stored with the extension **.pf** in the directory:

C:\Windows\Prefetch\

These files are extremely valuable in forensic investigations because they reveal program execution history, frequency of execution, and associated file paths.

1. Purpose of Prefetch Files

When a program is executed, Windows creates or updates a prefetch file to store:

- Required DLLs
- File dependencies
- Execution count
- Last execution time

- Application path

This improves future load performance.

2. Structure of Prefetch Files

The structure varies between Windows versions but commonly includes:

A. Header Section

Contains:

- Prefetch format version
 - Executable name
 - Hash of file path
 - File size
-

B. Execution Information

Includes:

- Last execution timestamp (FILETIME format)
 - Number of times program was run (RUN COUNT)
 - Volume information (volume serial number, creation timestamp)
-

C. File Metrics Section

Lists:

- DLL files loaded by the application
 - Resources accessed
 - File paths
-

D. Trace Chain Section

Contains run-time behavior of the executable.

E. Volume Information Section

Includes:

- Volume path
 - Serial number
 - Creation date of drive
-

3. Forensic Importance of Prefetch Files

A. Application Execution Evidence

Prefetch files prove:

- Which program was run
- When it was run last
- How many times it was run

This is crucial for identifying malicious programs.

Example:

- Malware.exe → RUN COUNT: 3 → PROVES execution
-

B. Linking Suspect Activity

Prefetch timestamps correlate with:

- Registry Run keys
- Event logs
- MFT timestamps

This helps reconstruct user activity.

C. Recovery of Deleted Prefetch Files

Even deleted .pf files may be recovered from:

- \$MFT
 - \$LogFile
 - Shadow copies
-

D. Volume Information Correlation

Prefetch files reveal the disk on which the program was stored — useful when removable media was used.

E. Anti-Forensic Detection

Attackers often delete prefetch files.
This itself becomes forensic evidence.

Conclusion

Prefetch files are critical forensic artifacts that reveal program execution history and help reconstruct user behavior. Their detailed metadata and timestamps make them one of the most valuable sources of evidence in Windows forensic investigations.

Q11. Explain analysis of Unallocated Space in Windows file systems.

Unallocated space refers to the area of a storage device that is not currently assigned to any active file but may contain remnants of previously deleted data. Forensic analysts examine unallocated space to recover deleted files, hidden data, fragments, and other digital artifacts.

1. What is Unallocated Space?

When a file is deleted:

- The file system removes its directory entry

- The underlying clusters are marked as “free”
- The actual data remains intact until overwritten

This area is known as **unallocated space**.

2. Sources of Data in Unallocated Space

A. Deleted Files

Files deleted from Recycle Bin remain in unallocated space.

B. Fragmented File Pieces

Parts of large files may remain scattered.

C. Partial Overwrites

Overwritten files may still contain residual data.

D. Previous Partitions

Old file systems may leave their structures untouched.

3. Forensic Techniques for Analyzing Unallocated Space

A. File Carving

Extracting files using headers and footers (e.g., JPEG FFD8/FFD9).

Tools:

- PhotoRec
 - Foremost
 - Scalpel
-

B. Keyword Searching

Searching for:

- Keywords
- Email addresses

- Document fragments

Tools such as FTK and X-Ways scan raw sectors for patterns.

C. Metadata Recovery

NTFS metadata such as:

- MFT entries
 - \$LogFile
 - \$UsnJrnl
- can reveal references to deleted files.
-

D. Entropy Analysis

Used to detect encrypted or compressed data hidden in unallocated space.

4. Forensic Importance

A. Recovery of Evidence

Deleted emails, images, documents, and chat logs can be recovered.

B. Discovery of Hidden or Obfuscated Data

Attackers may hide data in unallocated space.

C. Reconstruction of Timelines

Recovered fragments help reconstruct user activity.

D. Anti-Forensic Detection

Sudden wiping or wiping patterns indicate tampering.

5. Challenges

- SSDs may erase unallocated space due to TRIM
- Fragmentation complicates carving

- Overwritten data may be partially recoverable
-

Conclusion

Unallocated space is a treasure trove of digital evidence. By applying carving, metadata analysis, and keyword searches, investigators can recover deleted or hidden data essential for forensic examinations.

Q12. Explain the analysis of Program Execution Artifacts in Windows.

Program execution artifacts provide proof that an application or executable was launched on a Windows system. These artifacts help determine attacker activity, user behavior, malware execution, and timeline reconstruction.

Below are the most important artifacts used in forensic analysis.

1. Prefetch Files (.pf)

Location: C:\Windows\Prefetch\

Prefetch reveals:

- Program name
- Last execution time
- Number of runs
- DLL dependencies
- Associated file paths

Essential for confirming malware execution.

2. Windows Event Logs

Event IDs relevant to program execution:

- **4688** → New process created

- **4697** → Service installed
- **7036** → Service started/stopped

These logs confirm process activity.

3. ShimCache (AppCompatCache)

Location:

HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\AppCompatCache

Stores:

- Executable file paths
- File size
- Last modified time
- Execution evidence (not always definitive)

Helpful for malware detection.

4. AmCache

Location:

C:\Windows\AppCompat\Programs\Amcache.hve

Contains:

- Full file path
- SHA1 hash of executable
- First run time
- Program version

Very reliable for malware investigations.

5. UserAssist Keys

Location:

HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist

Stores:

- GUI-based program execution history
- Run count
- ROT13 encoded entries

Useful for identifying user-launched programs.

6. MUICache

Stores:

- Program display names
- Execution history

Helps identify unknown executables.

7. Jump Lists

Location:

C:\Users\<User>\AppData\Roaming\Microsoft\Windows\Recent\AutomaticDestinations\

Jump lists store:

- Program usage
 - Recently opened files
 - File paths associated with applications
-

8. Registry Run Keys (Persistence)

Common malware persistence keys:

- HKCU\...\Run
- HKLM\...\Run

Show which programs execute at startup.

9. Prefetch + Event Logs + Registry Correlation

A complete forensic reconstruction uses:

- Prefetch → Execution time & frequency
- Event Logs → Process creation & service changes
- Registry keys → Persistence and user activity
- MFT timestamps → File modifications

Together they prove execution beyond doubt.

Conclusion

Program execution artifacts are essential for confirming the launch of applications and malware on Windows systems. By analyzing Prefetch files, registry traces, Amcache, ShimCache, Jump Lists, and Event Logs, investigators can reconstruct user activities and attacker behavior with high accuracy.
